

INTRODUCTION TO MATLAB AND SIGNAL ANALYSIS

Objective: To get familiar with MATLAB and learn how to generate and visualize various basic signal waveform

Theory

Introduction

A **signal** is a function that carries information about a physical phenomenon or system. Signals are widely used in communication, control systems, instrumentation, audio processing, and many other engineering applications. Signal analysis involves studying the characteristics and behavior of signals to extract useful information.

Signals can be classified into two categories:

- **Continuous-Time Signals:** Defined at every instant of time and represented as $x(t)$.
- **Discrete-Time Signals:** Defined only at specific sampling instants and represented as $x[n]$.

MATLAB is a powerful numerical computing and visualization software widely used for signal generation, analysis, and processing. It provides built-in functions for generating various waveforms such as sine, cosine, square, and sawtooth waves, allowing users to analyze both continuous and discrete signals efficiently.

Types of Signals Used in the Experiment

Sine Wave

A sine wave is a periodic waveform commonly used to represent oscillatory phenomena. It is mathematically represented as:

$$x(t) = A \sin(2\pi ft) \quad (1)$$

where:

- A = Amplitude
- f = Frequency (Hz)
- t = Time (s)

Cosine Wave

A cosine wave is another fundamental periodic signal, differing from the sine wave by a phase shift of 90° . It is represented as:

$$x(t) = A \cos(2\pi ft) \quad (2)$$

where:

- A = Amplitude
- f = Frequency
- t = Time

Square Wave

A square wave alternates between positive and negative amplitude levels. MATLAB generates square waves using the `square()` function.

For a standard square wave:

$$x(t) = A \cdot \text{square}(2\pi ft) \quad (3)$$

For a square wave with duty cycle:

$$x(t) = A \cdot \text{square}(2\pi ft, D) \quad (4)$$

where:

- A = Amplitude
- f = Frequency
- D = Duty Cycle (%)

The duty cycle determines the percentage of one period during which the signal remains in the high state.

Sawtooth Wave

A sawtooth wave increases linearly and then drops sharply to its initial value. MATLAB provides the `sawtooth()` function for generating such waveforms.

$$x(t) = A \cdot \text{sawtooth}(2\pi ft, w) \quad (5)$$

where:

- A = Amplitude
- f = Frequency
- w = Width Parameter ($0 \leq w \leq 1$)

The width parameter controls the position of the waveform peak within one cycle.

Signal Parameters

The characteristics of a signal are determined by the following parameters:

Parameter	Description
Amplitude (A)	Maximum magnitude of the signal
Frequency (f)	Number of cycles completed per second (Hz)
Time (t)	Independent variable representing signal duration
Duty Cycle (D)	Percentage of a square wave period during which the signal remains high
Width Parameter (w)	Controls the shape and peak position of a sawtooth wave

Table 1: Signal Parameters

Continuous and Discrete Signal Representation

In MATLAB, continuous signals are visualized using the `plot()` function, which connects all data points to form a smooth waveform. Discrete signals are visualized using the `stem()` function, which displays individual sampled values without connecting lines.

The experiment involved generating and comparing continuous and discrete representations of sine, cosine, square, and sawtooth signals to understand their behavior and characteristics in signal analysis.

Applications of Signal Analysis

Signal analysis is used in:

- Digital communication systems
- Audio and speech processing
- Biomedical signal processing (ECG, EEG)
- Radar and sonar systems
- Control and automation systems

- Image and video processing

Thus, MATLAB serves as an effective platform for understanding the fundamentals of signal generation, visualization, and analysis in Digital Signal Processing (DSP).

MATLAB Codes

Question 1: Continuous and Discrete Sine and Cosine Waves

```

clc;
close all;
clear all;

A=5;
f=3;
t = 0:0.01:1;
x = A*sin(2*pi*f*t);
y = A*cos(2*pi*f*t);

subplot (2,2,1)
plot(t,x);
xlabel('time');
ylabel('amplitude');
title('continuous sinewave/ Himal Joshi, 28');
grid on ;

subplot (2,2,2)
plot(t,y);
xlabel('time');
ylabel('amplitude');
title('continuous coswave/ Himal Joshi, 28');
grid on ;

subplot (2,2,3)
stem(t,x);
xlabel('time');
ylabel('amplitude');
title('discrete sinewave/ Himal Joshi, 28');
grid on ;

subplot (2,2,4)

```

```

stem(t,y);
xlabel('time');
ylabel('amplitude');
title('discrete coswave/ Himal Joshi, 28');
grid on ;

```

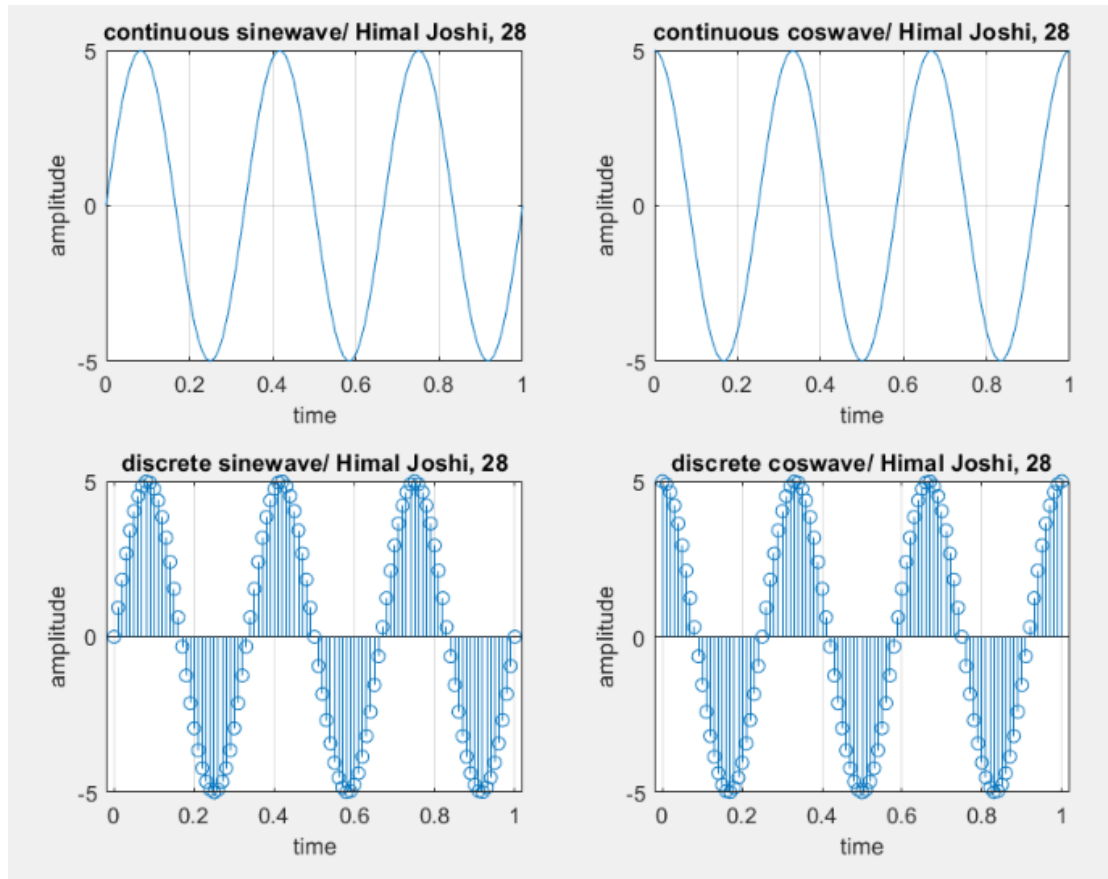


Figure 1: Continuous and Discrete Sine and Cosine Waves

Question 2: Continuous and Discrete Square Wave

```

clc;
close all;
clear all;

A=5;
f=3;
dutycycle =1;
t = 0:0.01:1;
x = A*square(2*pi*f*t* dutycycle);

subplot (2,2,1)
plot(t,x);

```

```

xlabel('time');
ylabel('amplitude');
title('continuous square wave/ Himal Joshi, 28');
grid on ;

```

```

subplot (2,2,2)
stem(t,x);
xlabel('time');
ylabel('amplitude');
title('discrete square wave/ Himal Joshi, 28');
grid on ;

```

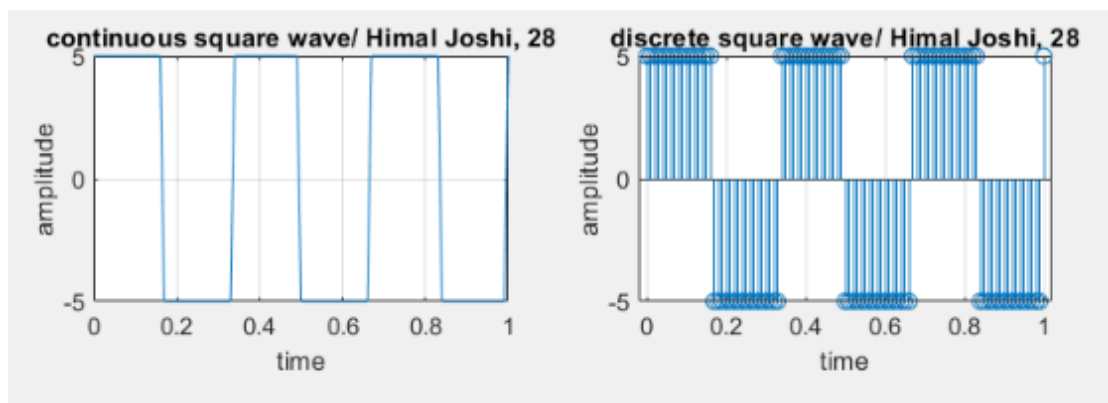


Figure 2: Continuous and Discrete Square Wave

Question 3: Square Wave with Duty Cycle

```

clc;
close all;
clear all;

A=5;
f=3;
t = 0:0.01:1;
x = A*square(2*pi*f*t, 75);

subplot (2,2,1)
plot(t,x);
xlabel('time');
ylabel('amplitude');
title('continuous duty square wave/ Himal Joshi, 28');
grid on ;

```

```

subplot (2,2,2)
stem(t,x);
xlabel('time');
ylabel('amplitude');
title('discrete duty square wave/ Himal Joshi, 28');
grid on ;

```

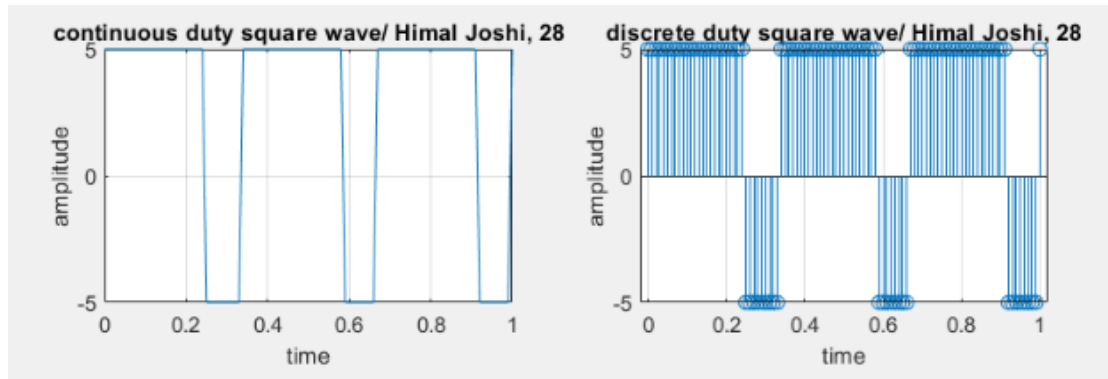


Figure 3: Square Wave with Duty Cycle

Question 4: Continuous and Discrete Sawtooth Wave

```

clc;
close all;
clear all;

A=5;
f=3;
t = 0:0.01:1;
x = A*sawtooth(2*pi*f*t,0.1);

subplot (2,2,1)
plot(t,x);
xlabel('time');
ylabel('amplitude');
title('continuous sawtooth wave/ Himal Joshi, 28');
grid on ;

subplot (2,2,2)
stem(t,x);
xlabel('time');
ylabel('amplitude');
title('discrete sawtooth wave/ Himal Joshi, 28');

```

```
grid on ;
```

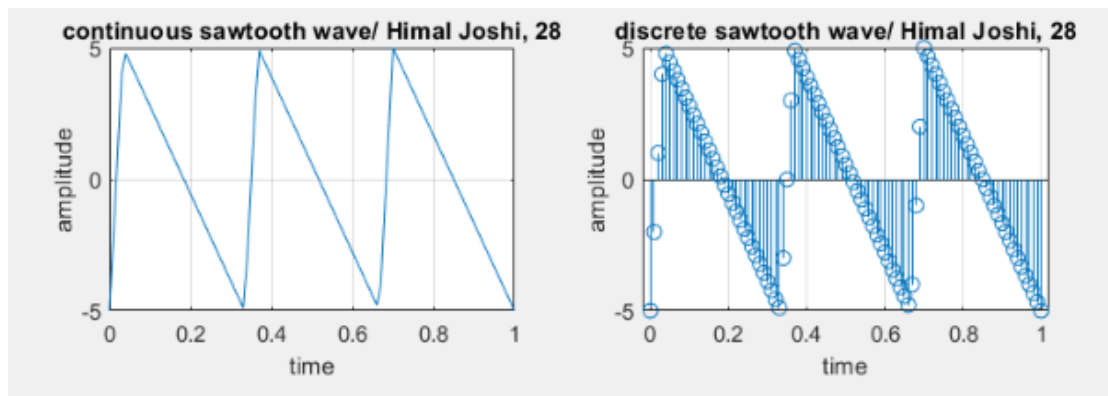


Figure 4: Continuous and Discrete Sawtooth Wave

Question 5: Sawtooth Waves with Different Width Parameters

```
clc;
close all;
clear all;
A=5;
f=3;
t = 0:0.01:1;
x = A*sawtooth(2*pi*f*t,0.2);
y = A*sawtooth(2*pi*f*t,0.5);
z = A*sawtooth(2*pi*f*t,0.7);
m = A*sawtooth(2*pi*f*t,0.9);

subplot (2,4,1)
plot(t,x);
xlabel('time');
ylabel('amplitude');
title('continuous sawtooth wave/ Himal Joshi, 28');
grid on ;
subplot (2,4,2)
stem(t,x);
xlabel('time');
ylabel('amplitude');
title('discrete sawtooth wave/ Himal Joshi, 28');
grid on ;

subplot (2,4,3)
plot(t,y);
```

```

xlabel('time');
ylabel('amplitude');
title('continuous sawtooth wave/ Himal Joshi, 28');
grid on ;
subplot (2,4,4)
stem(t,y);
xlabel('time');
ylabel('amplitude');
title('discrete sawtooth wave/ Himal Joshi, 28');
grid on ;

subplot (2,4,5)
plot(t,z);
xlabel('time');
ylabel('amplitude');
title('continuous sawtooth wave/ Himal Joshi, 28');
grid on ;
subplot (2,4,6)
stem(t,z);
xlabel('time');
ylabel('amplitude');
title('discrete sawtooth wave/ Himal Joshi, 28');
grid on ;

subplot (2,4,7)
plot(t,m);
xlabel('time');
ylabel('amplitude');
title('continuous sawtooth wave/ Himal Joshi, 28');
grid on ;
subplot (2,4,8)
stem(t,m);
xlabel('time');
ylabel('amplitude');
title('discrete sawtooth wave/ Himal Joshi, 28');
grid on ;

```

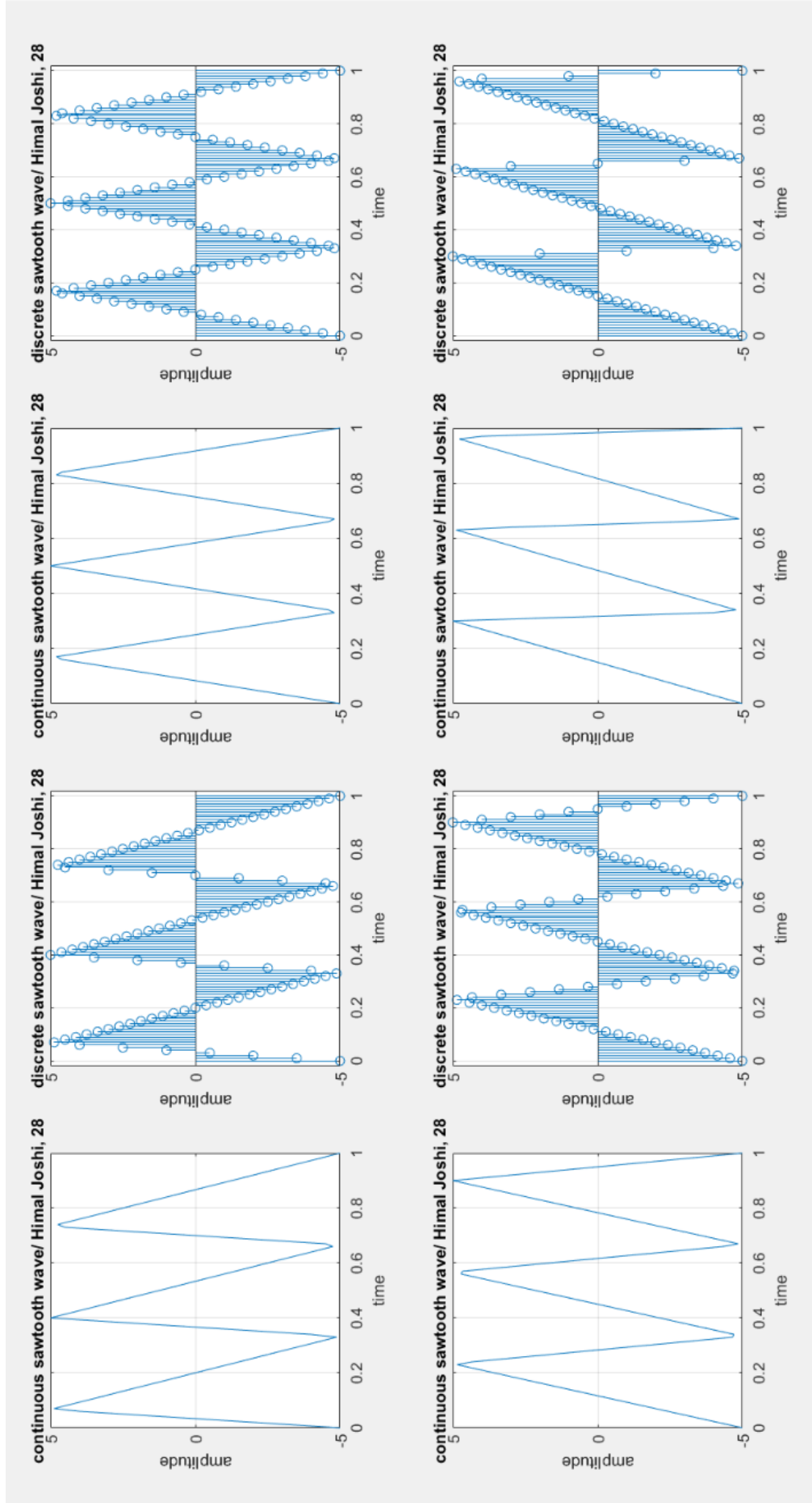


Figure 5: Sawtooth Waves with Different Width Parameters

Discussion

In this experiment, MATLAB was used to generate and analyze different types of signals, including sine, cosine, square, and sawtooth waves. Both continuous and discrete representations of the signals were observed using the `plot()` and `stem()` functions, respectively. The effects of varying parameters such as duty cycle in square waves and width parameter in sawtooth waves were also studied. The generated waveforms matched the expected theoretical behavior, demonstrating the effectiveness of MATLAB for signal visualization and analysis.

Conclusion

The experiment successfully introduced the basic features of MATLAB for signal generation and analysis. Continuous and discrete forms of sine, cosine, square, and sawtooth signals were generated and examined. The influence of duty cycle and width parameters on waveform shape was clearly observed. Therefore, MATLAB proved to be a useful tool for understanding fundamental concepts of Digital Signal Processing and signal visualization.